PROJECT WORKFLOW

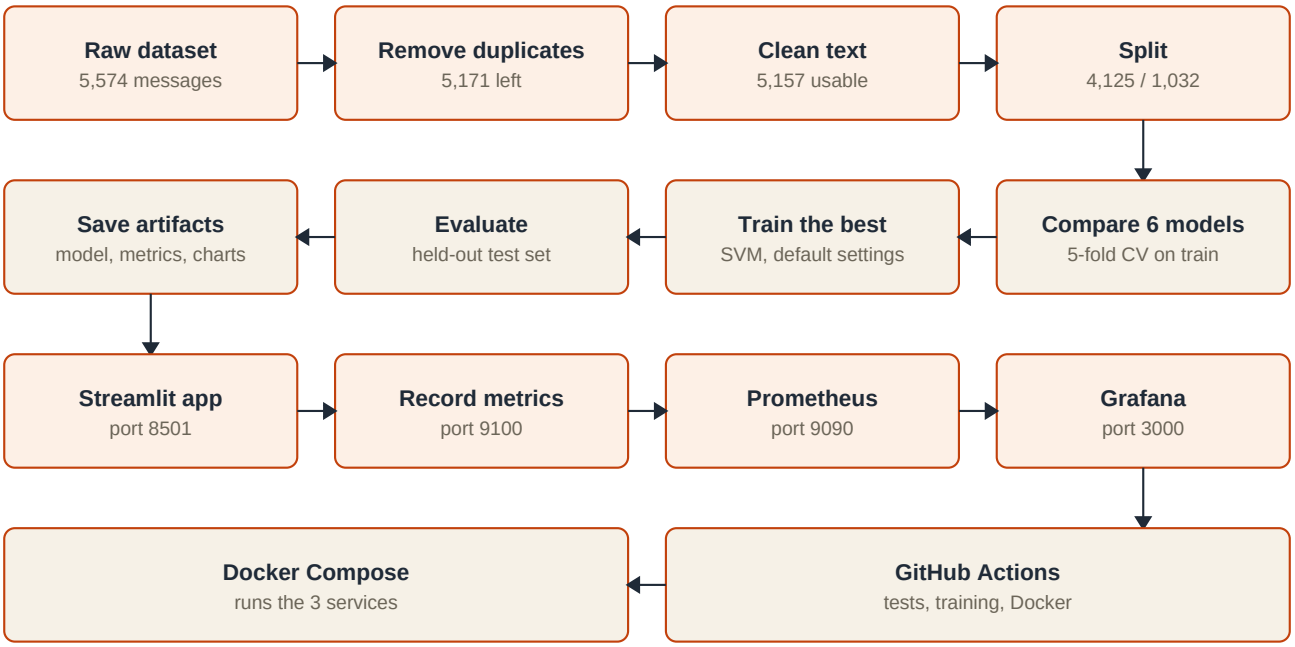
SMS Spam Detection

Every step in the repository, from the raw dataset to a monitored running app.

Author: Harvik · github.com/harvik07/sms-spam-dete

Goal	Classify a text message as spam or clean, and show which words drove the verdict.
Final model	Support Vector Machine on TF-IDF features, using default settings
Test results	Accuracy 97.9%, precision 99.1%, recall 84.0%, F1 0.909
Stack	Python, scikit-learn, NLTK, Streamlit, Prometheus, Grafana, Docker, GitHub Actions

The whole workflow on one page



The phases below follow this order. Each step names the file that does the work.

Phase	What happens	Steps
1. Data	Read the dataset, clean it and split it	1 to 5
2. Training	Build features, compare models and train the best one	6 to 8
3. Evaluation	Measure on unseen messages and save the results	9 to 11
4. App	Score a real message and show the verdict	12 to 15
5. Monitoring	Record, collect and display what the app is doing	16 to 19
6. Delivery	Package the project and check every push automatically	20 to 23

PHASE 1

Data

Step 1. Get the dataset

Where: `data/raw/SMSSpamCollection`

The project uses the SMS Spam Collection from the UCI Machine Learning Repository, created by Tiago A. Almeida and José María Gómez Hidalgo.

Nothing is downloaded when the code runs. The dataset file is stored in the repository, so training always uses exactly the same data and works offline.

Step 2. Load the file

Where: `spam_detector/data.py`

load_raw() reads the file as tab-separated text with two columns: the label and the message. Quote handling is turned off because messages contain stray quote marks, which would otherwise merge rows together.

Result: 5,574 messages, made up of 4,827 ham (normal) and 747 spam.

Step 3. Remove duplicates

Where: `spam_detector/data.py`

remove_duplicates() keeps the first copy of each repeated message. This prevents the same message from appearing in both the training and test data, which would inflate the score.

Result: 5,171 unique messages, 4,518 ham and 653 spam.

Step 4. Add the target and clean the text

Where: `spam_detector/preprocess.py, spam_detector/data.py`

add_features() sets the target to 0 for ham and 1 for spam, then runs clean_text() on every message. Here is what each stage does to one example:

Stage	What it does	Example output
Original	Raw message	URGENT! You have WON 2 free prizes
Tokenize	Lowercase, keep alphabetic words only	urgent you have won free prizes
Stop words	Remove common words such as 'you' and 'have'	urgent won free prizes
Stem	Reduce words to a root form with Porter stemming	urgent won free prize

14 messages have no usable words after cleaning and are dropped, leaving 5,157. About 12.6% of them are spam, so the data is imbalanced and accuracy alone would flatter the model.

Step 5. Split into training and test sets

Where: `spam_detector/data.py, spam_detector/config.py`

split() holds back 20% of the messages for the final test. The split is stratified, so both sets keep the same spam ratio, and the random seed is fixed at 42 so it can be repeated.

Result: 4,125 training messages and 1,032 test messages. The test set is not used again until Step 9.

PHASE 2

Training

Step 6. Turn text into numbers with TF-IDF

Where: `spam_detector/models.py`

Models need numbers, so TfidfVectorizer gives each word a weight. A word scores high when it is frequent in a message but rare across all messages, so 'prize' counts for more than an everyday word.

TF-IDF sits inside a scikit-learn Pipeline, so during cross-validation it learns only from the training folds. The final vocabulary has 5,466 words.

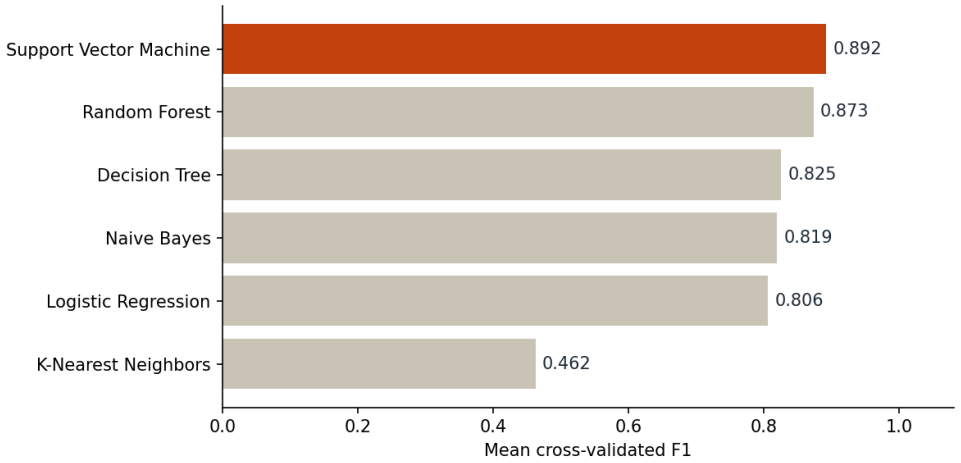
Step 7. Compare six models

Where: `spam_detector/models.py`

compare_models() runs 5-fold stratified cross-validation on the training set only. Each model trains on four folds and is scored on the fifth, five times over, and the scores are averaged.

All models use library default settings. **No hyperparameter tuning is performed.**

Model	Accuracy	Precision	Recall	F1
Support Vector Machine	0.975	0.986	0.814	0.892
Random Forest	0.971	0.984	0.785	0.873
Decision Tree	0.957	0.845	0.809	0.825
Naive Bayes	0.961	0.997	0.695	0.819
Logistic Regression	0.959	0.989	0.680	0.806
K-Nearest Neighbors	0.912	1.000	0.301	0.462



Step 8. Pick the winner and train it

Where: `spam_detector/pipeline.py`

Models are ranked by F1, which balances precision and recall and suits imbalanced data. The Support Vector Machine ranked first, so it is retrained on the full training set.

Run all of Steps 2 to 10 with one command: `python -m spam_detector.pipeline`

PHASE 3

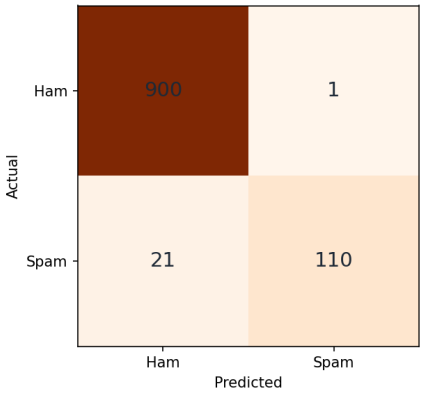
Evaluation

Step 9. Test on 1,032 unseen messages

Where: `spam_detector/evaluate.py`

evaluate() scores the trained model on the held-out test set, which played no part in choosing or training it.

Metric	Score	What it means here
Accuracy	97.9%	Share of all messages classified correctly
Precision	99.1%	When it says spam, how often it is right
Recall	84.0%	How much of the real spam it catches
F1 score	0.909	Balance of precision and recall



900 clean messages correctly passed

1 clean message wrongly flagged as spam

110 spam messages correctly caught

21 spam messages missed

The model is cautious: it almost never blocks a real message, but it lets about 1 in 6 spam messages through.

Step 10. Save the results

Where: `spam_detector/pipeline.py, spam_detector/plots.py`

The pipeline writes everything the later steps need:

File	Contents	Used by
models/spam_pipeline.joblib	Trained TF-IDF and model	App
models/metrics.json	Test scores and confusion matrix	App, Prometheus
reports/model_comparison.csv	Cross-validation scores	App
reports/figures/*.png	Model comparison and confusion matrix charts	App, this document

Step 11. Run the automated tests

Where: `tests/`

27 tests check the project with pytest. They cover text cleaning, data loading, model comparison, prediction, HTML escaping, metrics and the full Streamlit app.

Command: `python -m pytest`

PHASE 4

The app

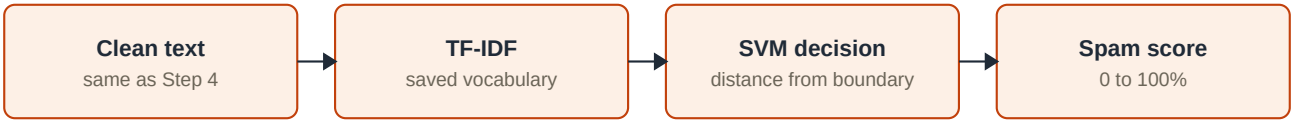
Step 12. Start the app

Where: `app.py, spam_detector/serve.py`

app.py loads the saved model once and reuses it for every visitor. In Docker, spam_detector/serve.py starts first so metrics are available before anyone opens the page.

Step 13. Score a message

When someone clicks Inspect message, SpamDetector.predict() runs the same steps as training:



Scores of 50% or more are labelled spam. The SVM gives a distance rather than a probability, so a sigmoid function converts it to a 0 to 100% scale. Treat it as a confidence indicator, not an exact probability.

Step 14. Explain the verdict

Where: `spam_detector/predict.py, ui/components.py`

For each word, the app removes that word and scores the message again. If the score falls, the word was pushing towards spam and appears in red. If it rises, the word appears in green.

Step 15. Show the three tabs

Where: `app.py, ui/`

Tab	What it does
Inspect	Scores one message and shows the verdict stamp, score bar and highlighted words
Batch	Scores many messages at once, one per line, in a table
Model report	Shows test scores, charts and the model comparison

PHASE 5

Monitoring

Step 16. Record metrics in the app

Where: `spam_detector/monitoring.py`

Each scored message updates counters and histograms in memory. A small web server inside the app publishes them at port 9100 on /metrics.

Metric	Measures
spam_requests_total	Messages scored, split into single and batch
spam_predictions_total	Verdicts, split into spam and clean
spam_rejected_requests_total	Empty submissions
spam_prediction_latency_seconds	Time to score each message
spam_score	Distribution of spam scores
spam_message_length_characters	Distribution of message lengths
spam_model_test_score	Accuracy, precision, recall and F1 from Step 9
spam_model_info	Name of the loaded model

Step 17. Collect metrics with Prometheus

Where: `monitoring/prometheus/prometheus.yml`

Prometheus fetches /metrics from the app every 15 seconds and stores the history.

Step 18. Watch for problems with alert rules

Where: `monitoring/prometheus/alerts.yml`

Alert	Fires when
SpamDetectorDown	The app cannot be reached for 1 minute
SlowPredictions	95th percentile latency stays above 1 second for 5 minutes
UnusualSpamShare	Over 80% of messages are flagged as spam for 15 minutes

These alerts appear in Prometheus. No email or chat notifications are set up.

Step 19. Display everything in Grafana

Where: `monitoring/grafana/`

Grafana loads its data source and dashboard automatically on startup. Open **SMS Spam Detector** at localhost:3000.

Section	Panels
Model quality	Accuracy, precision, recall, F1 gauges
Live traffic	Total requests, spam flagged, spam share, rejected requests, app status, request rate, verdicts over time
Performance and inputs	Latency (p50, p95, p99), spam score distribution, message length distribution

Important: the accuracy gauges show the fixed test results from Step 9, not live accuracy. Real messages arrive with no correct answer attached, so live accuracy cannot be measured. Request counts reset when the app restarts.

PHASE 6

Delivery

Step 20. Package the app with Docker

Where: `Dockerfile, .dockerignore`

The Dockerfile starts from Python 3.11, installs requirements.txt, and copies in the code, trained model and reports. It runs as a non-root user and includes a health check. The image serves the saved model and does not retrain.

Step 21. Run all services with Docker Compose

Where: `docker-compose.yml`

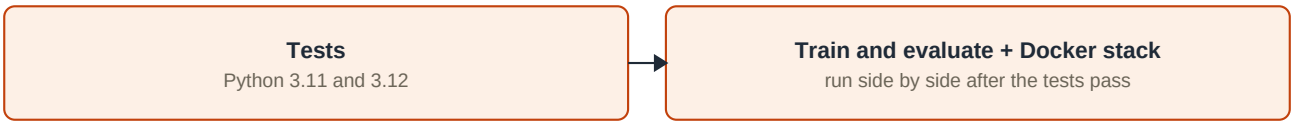
Service	Address	Role
app	localhost:8501	Streamlit app, with metrics on the internal network
prometheus	localhost:9090	Collects metrics; waits until the app is healthy
grafana	localhost:3000	Dashboard; default login admin / admin

Prometheus and Grafana data persists in named volumes across restarts. Change the Grafana password before exposing it beyond your own machine.

Step 22. Check every push with GitHub Actions

Where: `.github/workflows/ci.yml`

Each push to main runs the workflow. Training and Docker checks only start once the tests pass.



Job	What it verifies
Tests	All pytest tests pass on both Python versions
Train and evaluate	The full pipeline runs from scratch, accuracy exceeds 90%, and the report is uploaded
Docker and monitoring stack	Compose and Prometheus configs are valid, all services start, Grafana loads the dashboard and Prometheus reads the app's metrics

Step 23. Run it yourself

Train the model and run the tests:

```
pip install -r requirements-dev.txt
python -m spam_detector.pipeline
python -m pytest
```

Run only the app:

```
streamlit run app.py
```

Run the full stack:

```
docker compose up -d --build
```

REFERENCE

Repository map

Path	Purpose
app.py	Streamlit app
spam_detector/config.py	Paths and settings
spam_detector/data.py	Load, deduplicate, label, split
spam_detector/preprocess.py	Text cleaning
spam_detector/models.py	Candidate models and comparison
spam_detector/evaluate.py	Test set scoring
spam_detector/plots.py	Report charts
spam_detector/pipeline.py	Runs the training workflow
spam_detector/predict.py	Scoring and word signals
spam_detector/monitoring.py	Prometheus metrics
spam_detector/serve.py	Container entrypoint
ui/	Page styling and HTML components
tests/	Automated tests
data/raw/	Dataset
models/, reports/	Training outputs
monitoring/	Prometheus and Grafana configuration
.streamlit/	App theme
.github/workflows/ci.yml	GitHub Actions workflow
Dockerfile, docker-compose.yml	Containers
docs/project-workflow.pdf	This document